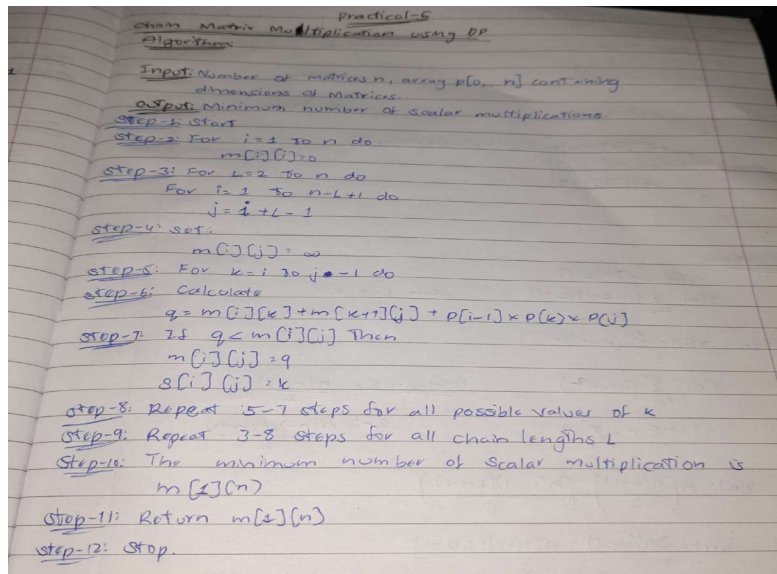


PRACTICAL-06

AIM: Implementation of chain matrix multiplication using dynamic programming.

ALGORITHM:



PROGRAM:

```
#include <iostream>

#include <limits>

using namespace std;

int main()
{
    int n;

    cout << "Enter number of matrices: ";

    cin >> n;

    int p[n + 1];

    cout << "Enter matrix dimensions: ";

    for (int i = 0; i <= n; i++)

        cin >> p[i];

    int m[n + 1][n + 1];
```

```
for (int i = 1; i <= n; i++)  
    m[i][i] = 0;  
  
for (int L = 2; L <= n; L++)  
{  
    for (int i = 1; i <= n - L + 1; i++)  
    {  
        int j = i + L - 1;  
        m[i][j] = INT_MAX;  
        for (int k = i; k < j; k++)  
        {  
            int q = m[i][k]  
                + m[k + 1][j]  
                + p[i - 1] * p[k] * p[j];  
  
            if (q < m[i][j])  
                m[i][j] = q;  
        }  
    }  
}  
  
cout << "Minimum number of scalar multiplications: "  
    << m[1][n];  
  
return 0;  
}
```

OUTPUT:

Output

```
Enter number of matrices: 3  
Enter matrix dimensions: 10  
20  
30  
40  
Minimum number of scalar multiplications: 18000
```

TIME COMPLEXITY:

Time Complexity

```

for l = 2 to n
  for i = 1 to n-l+1
    for k = i to j-1
      q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]
      if (q < m[i][j])
        m[i][j] = q
    
```

→ n-1 times
→ (n-l+1) times
→ (k-i+1) times
→ 3 times
→ 3 times

calculations:

$$T(n) = \sum_{l=2}^{n-1} \sum_{i=1}^{n-l+1} \sum_{k=i}^{j-1} (n-l+1)$$

when $k=2, n=1$
 $k=n, n=n-1$

Therefore, $T(n) = \sum_{l=2}^{n-1} (n-l+1) \times (n-l+1) \times (n-l+1)$

Formula: $\sum_{r=1}^n r = \frac{n(n+1)}{2}$

$$T(n) = \sum_{l=2}^{n-1} (n-l+1)^3$$

$$T(n) = \sum_{l=2}^{n-1} (n-l+1)^3 = \sum_{r=1}^{n-1} r^3$$

$$T(n) = \frac{n(n-1)(n+1)(2n-1)}{6}$$

common den: $T(n) = \frac{3n^4(n-1) - n(n-1)(2n-1)}{6}$

$$T(n) = \frac{n(n-1)(3n^3 - (2n-1))}{6}$$

$$T(n) = \frac{n(n-1)(n^3 + 1)}{6}$$

$$T(n) = \frac{n(n^4 - 1)}{6} \Rightarrow T(n) = \frac{n^5 - n}{6}$$

$T(n) = O(n^5)$

Best case: even if first k gives the minimum

```

for l = 2 to n
  for i = 1 to n-l+1
    for k = i to j-1
      //
    
```

So, $T(n) = O(n^3)$

All possible k values are still checked.

Average case:

```

for l = 2 to n
  for i = 1 to n-l+1
    for k = i to j-1
      //
    
```

$T(n) = O(n^3)$

The algorithm checks all possible split positions

Worst case:

```

for l = 2 to n
  for i = 1 to n-l+1
    for k = i to j-1
      //
    
```

$T(n) = O(n^3)$

Even when the ~~minimum~~ minimum is found at the ~~last~~ last split, all k values are checked.

Best = Average = Worst = $O(n^3)$

CONCLUSION:

Matrix Chain Multiplication using Dynamic Programming finds the minimum number of scalar multiplications efficiently. It avoids repeated calculations by storing previous results in a DP table. The time complexity is $O(n^3)$ and the space complexity is $O(n^2)$.



Marwadi
University
Marwadi Chandarana Group



FACULTY OF ENGINEERING AND TECHNOLOGY

Department of Computer Engineering

01AI0506 – Design and Analysis of Algorithm – Lab Manual
